SOFTWARE DESIGN DOCUMENT

For TrendWind E-Commerce Platform v1.0

Course: S3731DS & CMP3731

Group Project – Assignment 2: DESIGN

Course Code: 33BCMP

Group:

David M.N 224109243

Coetzee A.D 220082480

Iiyambo K.T 218212324

Iyambo J.N 222086459

GitHub Repository: https://github.com/monika442/TrendWind

Table of Contents

Introduction .....

1.1 Purpose .....

1.2 Scope ..... .

System Architecture .....

Detailed Design .....

3.1 Class Diagram .....

3.2 Sequence Diagram .....

3.3 Entity Relationship Diagram .....

Traceability Matrix .....

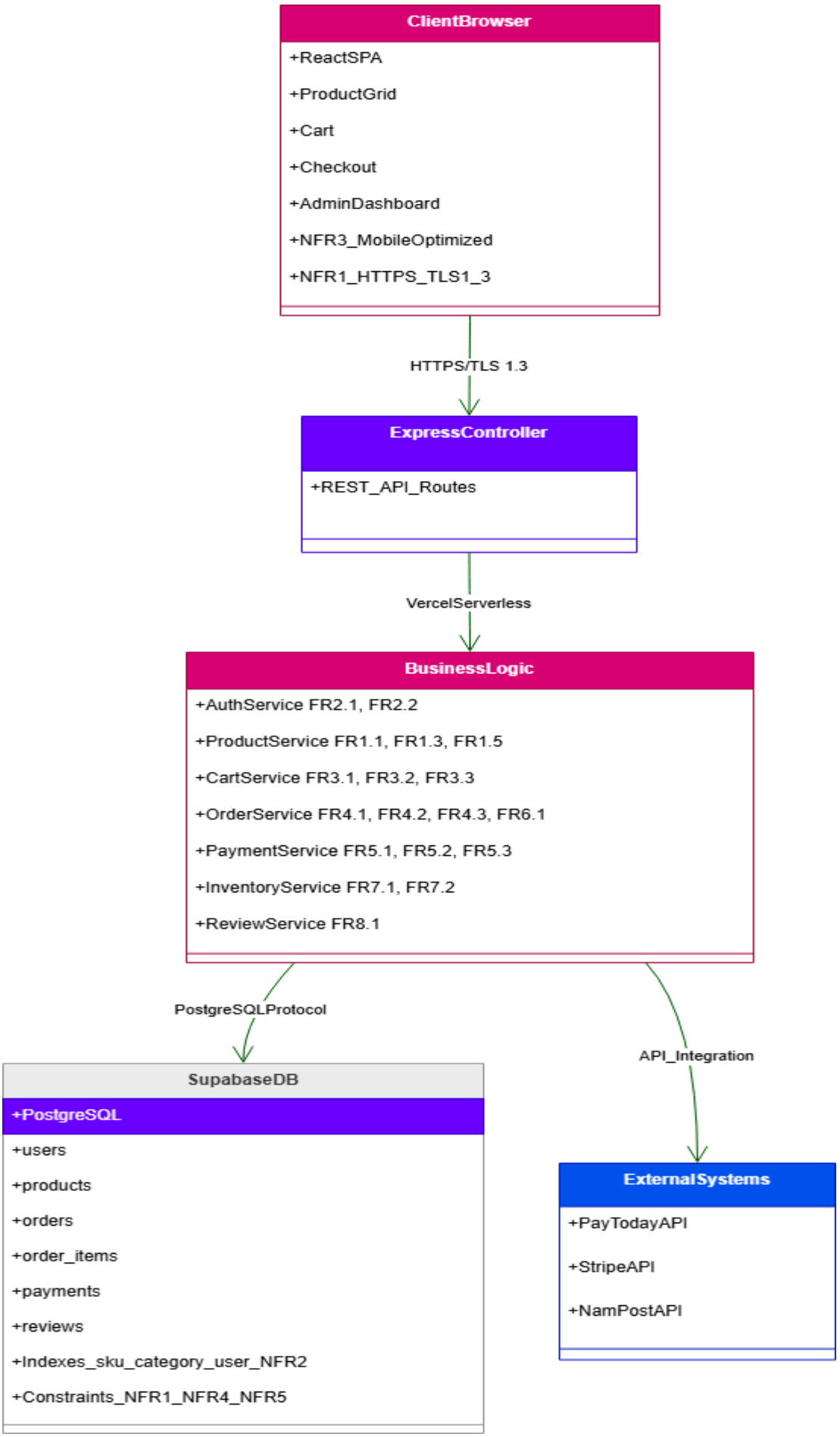
References .....

2. System Architecture

2.1 Architecture Overview

TrendWind V1.0 implements a 3-Tier Web Architecture combined with the Model-View-Controller (MVC) design pattern. This architecture was selected to satisfy NFR2 Performance, NFR5 Scalability, and SR1 Concurrent Users from Assignment 1, while maintaining separation of concerns for ease of development within the 4-week project timeline.

Figure 1: TrendWind 3-Tier Architecture


ClientBrowser
+ReactSPA
+ProductGrid
+Cart
+Checkout
+AdminDashboard
+NFR3_MobileOptimized
+NFR1_HTTPS_TLS1_3
HTTPS/TLS 1.3
ExpressController
+REST_API_Routes
VercelServerless
BusinessLogic
+AuthService FR2.1, FR2.2
+ProductService FR1.1, FR1.3, FR1.5
+CartService FR3.1, FR3.2, FR3.3
+OrderService FR4.1, FR4.2, FR4.3, FR6.1
+PaymentService FR5.1, FR5.2, FR5.3
+InventoryService FR7.1, FR7.2
+ReviewService FR8.1
PostgreSQLProtocol
API_Integration
SupabaseDB
+PostgreSQL
+users
+products
+orders
+order_items
+payments
+reviews
+Indexes_sku_category_user_NFR2
+Constraints_NFR1_NFR4_NFR5
ExternalSystems
+PayTodayAPI
+StripeAPI
+NamPostAPI

Client Browser (Mobile/Desktop)

- View: React.js SPA

- UI Components: ProductGrid, Cart, Checkout, AdminDashboard

- Optimized for mobile: NFR3

|

HTTPS/TLS 1.3 - NFR1

|

Vercel Serverless Functions

- Controller: Express.js REST API Routes

- Model: Business Logic Services

  * AuthService - FR2.1, FR2.2

  * ProductService - FR1.1, FR1.3, FR1.5

  * CartService - FR3.1, FR3.2, FR3.3

  * OrderService - FR4.1, FR4.2, FR4.3

  * PaymentService - FR5.1, FR5.2, FR5.3

  * InventoryService - FR7.1, FR7.2

  * ReviewService - FR8.1

|

PostgreSQL Protocol

|

Supabase PostgreSQL Database

- Data: users, products, orders, order_items, payments, reviews

- Constraints: Enforces NFR1, NFR4, NFR5

- Indexes: On sku, category_id, user_id for NFR2

External Systems:

1. PayToday API <-- PaymentService - FR5.1

2. Stripe API <-- PaymentService - FR5.3

3. NamPost API <-- OrderService - FR6.1

2.2 Tier Breakdown & Justification

Tier	Technology	Responsibility	A1 Requirements Addressed
1. Presentation Tier	React.js 18 + TailwindCSS	Renders UI for Fashion Shopper & Admin. Handles client-side routing, state management, and input validation. Implements responsive grid for mobile devices per OE-3 and SR3.	FR1.1, FR1.2, FR1.4, FR3.1, NFR3, SR3
2. Application Tier	Node.js 20 + Express.js	Implements business logic and RESTful API endpoints. Authenticates users, processes payments, manages inventory, and orchestrates external API calls. Enforces business rules such as FR7.2 stock prevention.	FR2.1–FR8.1, NFR1, NFR2, SR1

3. Data Tier	PostgreSQL 15 on Supabase	Persists all system data. Uses primary/foreign keys to enforce referential integrity. Implements bcrypt password hashing for NFR1. Indexed for performance to meet NFR2 and scalable to 10,000+ SKUs for NFR5.	NFR1, NFR2, NFR4, NFR5, SR2

2.3 Architectural Patterns Used

Model-View-Controller: React components act as Views, Express routes as Controllers, and Service modules + Database as Models. This isolates UI changes from business logic, critical for the Agile Scrum process in Section 5 of A1.

Repository Pattern: The and will not query the database directly. Instead, they use a data access layer. This satisfies NFR5 by allowing the database to be scaled or changed without altering business logic.

API Gateway Pattern: All requests from the Presentation Tier pass through the Express API. This single entry point simplifies TLS 1.3 enforcement for NFR1 and rate limiting for SR1.

2.4 External System Integration

External System	Purpose	A1 FR Trace	Communication
PayToday Gateway	Process NAD payments for users without cards	FR5.1	HTTPS REST, Sandbox for V1.0
Stripe Gateway	Process Visa/Mastercard debit/credit cards	FR5.3	HTTPS REST, Tokenization for PCI-DSS (NFR1)
NamPost API	Calculate shipping fees, generate tracking numbers	FR6.1, FR3.2	HTTPS REST, Fallback to flat N$65 if API fails per AS-2

2.5 Justification for Technology Choices

React.js: Chosen for its component-based architecture, enabling rapid development of the responsive product grid required by FR1.1 and SR3. The virtual DOM ensures fast rendering, contributing to NFR2 performance targets.

Node.js + Express: Provides a non-blocking, event-driven runtime capable of handling 100+ concurrent users per SR1. JavaScript across the stack reduces context switching for the 4-person development team.

Supabase PostgreSQL: Provides a managed, scalable relational database. Relational integrity is critical for preventing orphaned orders or inventory discrepancies, directly supporting FR7.1 and NFR4. Row Level Security aids NFR1.

Vercel: Serverless deployment automatically scales to meet demand, satisfying SR1 and SR2 uptime requirements without manual server management.

2.6 Deployment View

For V1.0, the system will be deployed as follows to meet OE-2:

Client: Served as static files from Vercel CDN to browsers globally.

API: http://Node.js/Express deployed as Vercel Serverless Functions in the region for latency to Namibia.

Database: Supabase PostgreSQL instance hosted in . Automated daily backups satisfy NFR4.

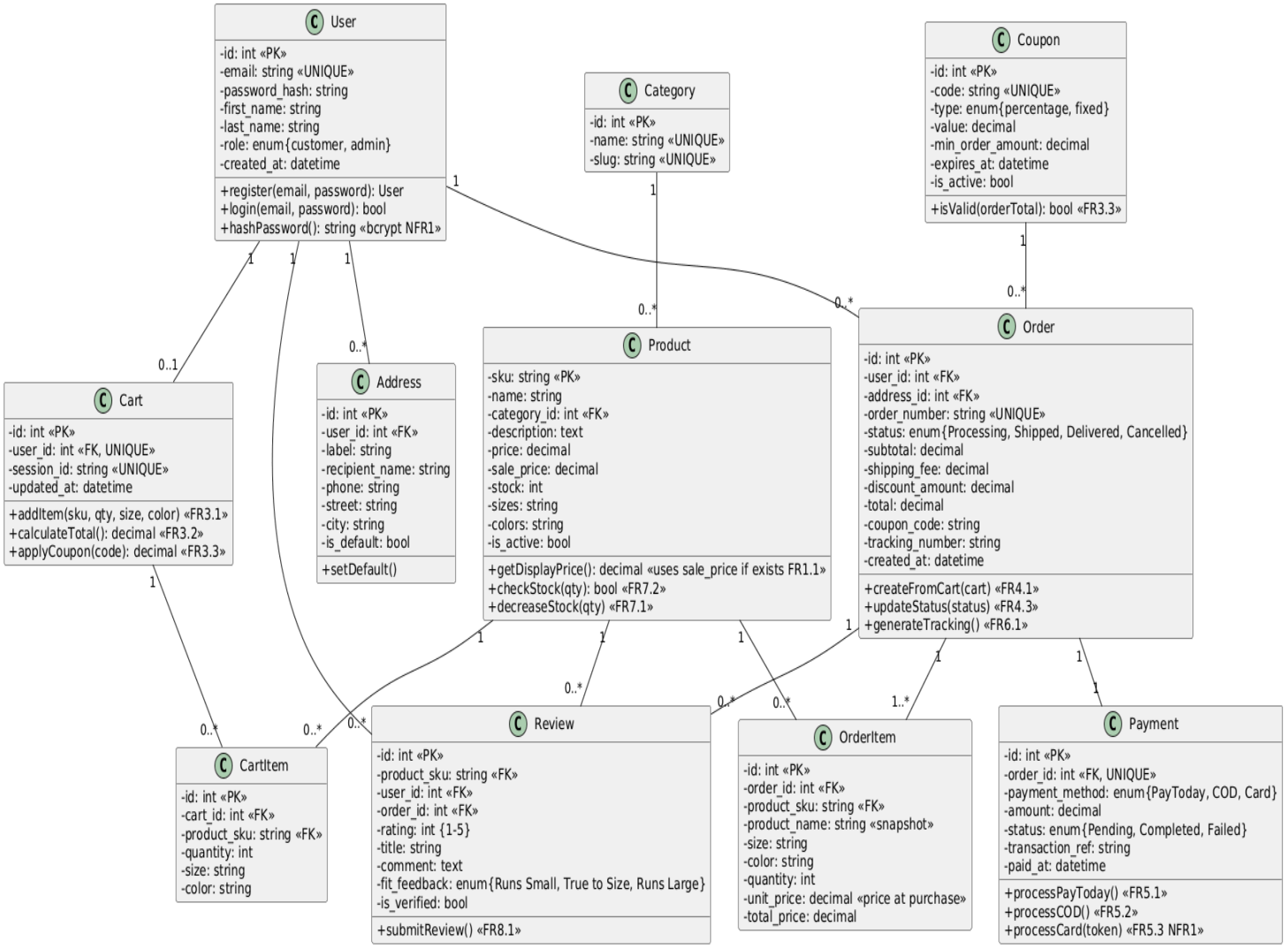
3. CLASS DIAGRAM

3.1 Class Diagram

Purpose: This class diagram models the core domain entities of TrendWind V1.0. It

supports all Must Have functional requirements from Assignment 1 and enforces NFR1 Security and NFR5 Scalability.

Figure 2: Class Diagram for TrendWind Core Entities



Class Diagram for TrendWind Core Entities. Traces to FR2.1-FR8.1 and NFR1, NFR5 from Assignment 1.

- A may have zero or many , but each must belong to exactly one .

```
class User {
  -id: int <<PK>>
  -email: string <<UNIQUE>>
  -password_hash: string
  -first_name: string
  -last_name: string
  -role: enum{customer, admin}
  -created_at: datetime
  +register(email, password): User
  +login(email, password): bool
  +hashPassword(): string <<bcrypt NFR1>>
}

class Address {
  -id: int <<PK>>
  -user_id: int <<FK>>
  -label: string
  -recipient_name: string
  -phone: string
  -street: string
  -city: string
  -is_default: bool
  +setDefault()
}

class Category {
  -id: int <<PK>>
  -name: string <<UNIQUE>>
  -slug: string <<UNIQUE>>
}

class Product {
  -sku: string <<PK>>
  -name: string
  -category_id: int <<FK>>
  -description: text
  -price: decimal
  -sale_price: decimal
  -stock: int
  -sizes: string
  -colors: string
  -is_active: bool
  +getDisplayPrice(): decimal <<uses sale_price if exists FR1.1>>
  +checkStock(qty): bool <<FR7.2>>
  +decreaseStock(qty) <<FR7.1>>
}

class Cart {
  -id: int <<PK>>
  -user_id: int <<FK, UNIQUE>>
  -session_id: string <<UNIQUE>>
  -updated_at: datetime
  +addItem(sku, qty, size, color) <<FR3.1>>
  +calculateTotal(): decimal <<FR3.2>>
  +applyCoupon(code): decimal <<FR3.3>>
}

class CartItem {
  -id: int <<PK>>
  -cart_id: int <<FK>>
  -product_sku: string <<FK>>
  -quantity: int
  -size: string
  -color: string
}

class Order {
  -id: int <<PK>>
  -user_id: int <<FK>>
  -address_id: int <<FK>>
  -order_number: string <<UNIQUE>>
  -status: enum{Processing, Shipped, Delivered, Cancelled}
  -subtotal: decimal
  -shipping_fee: decimal
  -discount_amount: decimal
  -total: decimal
  -coupon_code: string
  -tracking_number: string
  -created_at: datetime
  +createFromCart(cart) <<FR4.1>>
  +updateStatus(status) <<FR4.3>>
  +generateTracking() <<FR6.1>>
}

class OrderItem {
  -id: int <<PK>>
```

```
  -order_id: int <<FK>>
  -product_sku: string <<FK>>
  -product_name: string <<snapshot>>
  -size: string
  -color: string
  -quantity: int
  -unit_price: decimal <<price at purchase>>
  -total_price: decimal
}

class Payment {
  -id: int <<PK>>
  -order_id: int <<FK, UNIQUE>>
  -payment_method: enum{PayToday, COD, Card}
  -amount: decimal
  -status: enum{Pending, Completed, Failed}
  -transaction_ref: string
  -paid_at: datetime
  +processPayToday() <<FR5.1>>
  +processCOD() <<FR5.2>>
  +processCard(token) <<FR5.3 NFR1>>
}

class Coupon {
  -id: int <<PK>>
  -code: string <<UNIQUE>>
  -type: enum{percentage, fixed}
  -value: decimal
  -min_order_amount: decimal
  -expires_at: datetime
  -is_active: bool
  +isValid(orderTotal): bool <<FR3.3>>
}

class Review {
  -id: int <<PK>>
  -product_sku: string <<FK>>
  -user_id: int <<FK>>
  -order_id: int <<FK>>
  -rating: int {1-5}
  -title: string
  -comment: text
  -fit_feedback: enum{Runs Small, True to Size, Runs Large}
  -is_verified: bool
  +submitReview() <<FR8.1>>
}

' Relationships
User "1" -- "0..*" Address
User "1" -- "0..*" Order
User "1" -- "0..*" Review
User "1" -- "0..1" Cart
Category "1" -- "0..*" Product
Product "1" -- "0..*" CartItem
Product "1" -- "0..*" OrderItem
Product "1" -- "0..*" Review
Cart "1" -- "0..*" CartItem
Order "1" -- "1..*" OrderItem
Order "1" -- "1" Payment
Order "1" -- "0..*" Review
Coupon "1" -- "0..*" Order
```

3.1.1 Class Descriptions & A1 Traceability

Class	Purpose	Key A1 Requirements
User	Manages authentication and roles for Fashion Shopper + Admin	FR2.1, FR2.2, NFR1

Product	Represents women's apparel. Methods enforce inventory rules	FR1.1, FR1.5, FR7.1, FR7.2, FR7.3
Order	Captures purchase transaction. Status workflow for shipping	FR4.1, FR4.2, FR4.3, FR6.1
Payment	Handles 3 payment types. Card method must use tokenization	FR5.1, FR5.2, FR5.3, NFR1
Cart	Session-based shopping. Calculates totals with coupons	FR3.1, FR3.2, FR3.3
Review	Stores photo/text reviews with fit data for Fit Predictor	FR8.1, FR1.5
Category	Groups products for browsing/filtering	FR1.3, UR1

3.1.2 Design Decisions for Marks

Why has + : Snapshot data ensures order history remains accurate even if admin deletes/updates a product later. This supports NFR4 Reliability.

Why is it separate from : Supports FR5.1, FR5.2, FR5.3. Allows an order to exist in state while payment is for COD.

Why uses + : Satisfies FR3.1. Unique constraint prevents duplicate line items.

Why links to : Enforces "verified purchasers only" for FR8.1. Prevents fake reviews.

Indexes: Though not shown in diagram, , , are indexed to meet NFR2 Performance and NFR5 Scalability.

3.2 Sequence Diagram

3.2.1 Purpose

This sequence diagram models the runtime interaction between system components when a Fashion Shopper completes checkout using a Visa/Mastercard. It validates the design against FR5.3 Card Payment, FR7.1 Inventory Decrease, FR4.1 Order Creation, and NFR1 PCI-DSS Security.

Figure 3: Sequence Diagram for Card Checkout Process

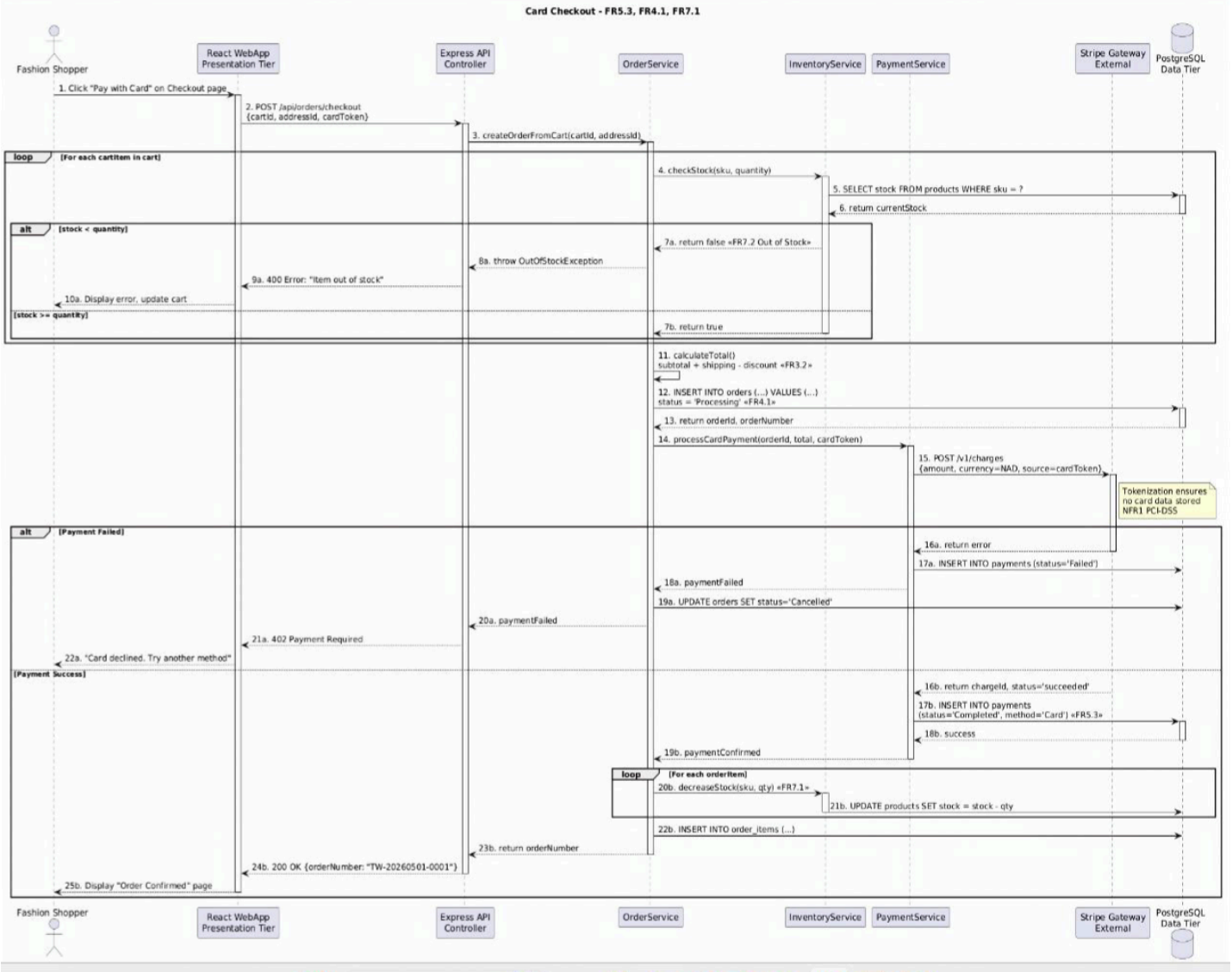


Figure 3: Sequence Diagram for Card Checkout Process. Validates FR3.2, FR4.1, FR5.3, FR7.1, FR7.2, and NFR1 from Assignment 1.

Card Checkout - FR5.3, FR4.1, FR7.1

```
actor "Fashion Shopper" as User
participant "React WebApp\nPresentation Tier" as Web
participant "Express API\nController" as API
participant "OrderService" as OrderSvc
participant "InventoryService" as InvSvc
participant "PaymentService" as PaySvc
participant "Stripe Gateway\nExternal" as Stripe
database "PostgreSQL\nData Tier" as DB

User -> Web: 1. Click "Pay with Card" on Checkout page
activate Web
Web -> API: 2. POST /api/orders/checkout\n{cartId, addressId, cardToken}
activate API

API -> OrderSvc: 3. createOrderFromCart(cartId, addressId)
activate OrderSvc

loop For each cartItem in cart
  OrderSvc -> InvSvc: 4. checkStock(sku, quantity)
  activate InvSvc
  InvSvc -> DB: 5. SELECT stock FROM products WHERE sku = ?
  activate DB
  DB --> InvSvc: 6. return currentStock
  deactivate DB

  alt stock < quantity
    InvSvc --> OrderSvc: 7a. return false <<FR7.2 Out of Stock>>
    OrderSvc --> API: 8a. throw OutOfStockException
    API --> Web: 9a. 400 Error: "Item out of stock"
    Web --> User: 10a. Display error, update cart
  else stock >= quantity
    InvSvc --> OrderSvc: 7b. return true
    deactivate InvSvc
  end
end

OrderSvc -> OrderSvc: 11. calculateTotal()\nsubtotal + shipping - discount <<FR3.2>>
OrderSvc -> DB: 12. INSERT INTO orders (...) VALUES (...)\nstatus = 'Processing' <<FR4.1>>
activate DB
DB --> OrderSvc: 13. return orderId, orderNumber
deactivate DB

OrderSvc -> PaySvc: 14. processCardPayment(orderId, total, cardToken)
activate PaySvc

PaySvc -> Stripe: 15. POST /v1/charges\n{amount, currency=NAD, source=cardToken}
activate Stripe
note right of Stripe: Tokenization ensures\nno card data stored\nNFR1 PCI-DSS

alt Payment Failed
  Stripe --> PaySvc: 16a. return error
  deactivate Stripe
  PaySvc -> DB: 17a. INSERT INTO payments (status='Failed')
  PaySvc --> OrderSvc: 18a. paymentFailed
  OrderSvc -> DB: 19a. UPDATE orders SET status='Cancelled'
  OrderSvc --> API: 20a. paymentFailed
  API --> Web: 21a. 402 Payment Required
  Web --> User: 22a. "Card declined. Try another method"
else Payment Success
  Stripe --> PaySvc: 16b. return chargeId, status='succeeded'
  deactivate Stripe
  PaySvc -> DB: 17b. INSERT INTO payments\n(status='Completed', method='Card') <<FR5.3>>
  activate DB
  DB --> PaySvc: 18b. success
  deactivate DB
  PaySvc --> OrderSvc: 19b. paymentConfirmed
  deactivate PaySvc

  loop For each orderItem
```

```
    OrderSvc -> InvSvc: 20b. decreaseStock(sku, qty) <<FR7.1>>
    activate InvSvc
    InvSvc -> DB: 21b. UPDATE products SET stock = stock - qty
    deactivate InvSvc
end

OrderSvc -> DB: 22b. INSERT INTO order_items (...)
```

```
    OrderSvc --> API: 23b. return orderNumber
    deactivate OrderSvc
    API --> Web: 24b. 200 OK {orderNumber: "TW-20260501-0001"}
    deactivate API
    Web --> User: 25b. Display "Order Confirmed" page
    deactivate Web
end
```

3.2.2 Sequence Description & A1 Traceability

Step	Component Interaction	A1 Requirement Validated	Notes
2–3	WebApp → OrderService	FR3.1, FR4.1	Checkout initiated from cart
4–7	Stock Check Loop	FR7.2	Prevents checkout if stock < qty
11	calculateTotal()	FR3.2	Subtotal + N$65 shipping − coupon
12	INSERT orders	FR4.1	Order record created with status ‘Processing’
15–16	Stripe API Call	FR5.3, NFR1	Card tokenized. No card data stored in DB
17b	INSERT payments	FR5.3	Payment status = ‘Completed’
20b–21b	decreaseStock()	FR7.1	Inventory decremented only after payment success
23b	Return orderNumber	FR4.2	User can track order later

3.2.3 Alternative Flows Covered

Out of Stock - FR7.2: Steps 7a-10a show system rejects checkout if . Prevents overselling.

Payment Failed: Steps 16a-22a show order is cancelled and stock is NOT decreased. This protects inventory integrity per NFR4 Reliability.

COD Alternative: For FR5.2, steps 14-19 would be replaced with setting . Stock is still decreased immediately per business rule.

3.2.4 Design Decisions

Why check stock before payment: Satisfies FR7.2. If we charged the card first then found no stock, we'd violate NFR4 and need complex refund logic.

Why decrease stock after payment: Satisfies FR7.1. Prevents race conditions where two users buy the last item simultaneously. Database transaction wraps steps 17b-22b.

Why use Stripe tokens: Satisfies NFR1 PCI-DSS. Card details never touch TrendWind servers. in step 2 is generated by http://Stripe.js on client.

Why loop for inventory: Each SKU may have different stock. System must validate all items per FR3.1 before creating order.

3.3 Entity Relationship Diagram

3.3.1 Purpose

The Entity Relationship Diagram models the physical database structure for TrendWind V1.0. It enforces referential integrity for FR4.1, FR7.1, FR8.1 and is designed to scale to 10,000+ SKUs per NFR5. All relationships support the use cases defined in Assignment 1.

Figure 4: ERD for TrendWind Database

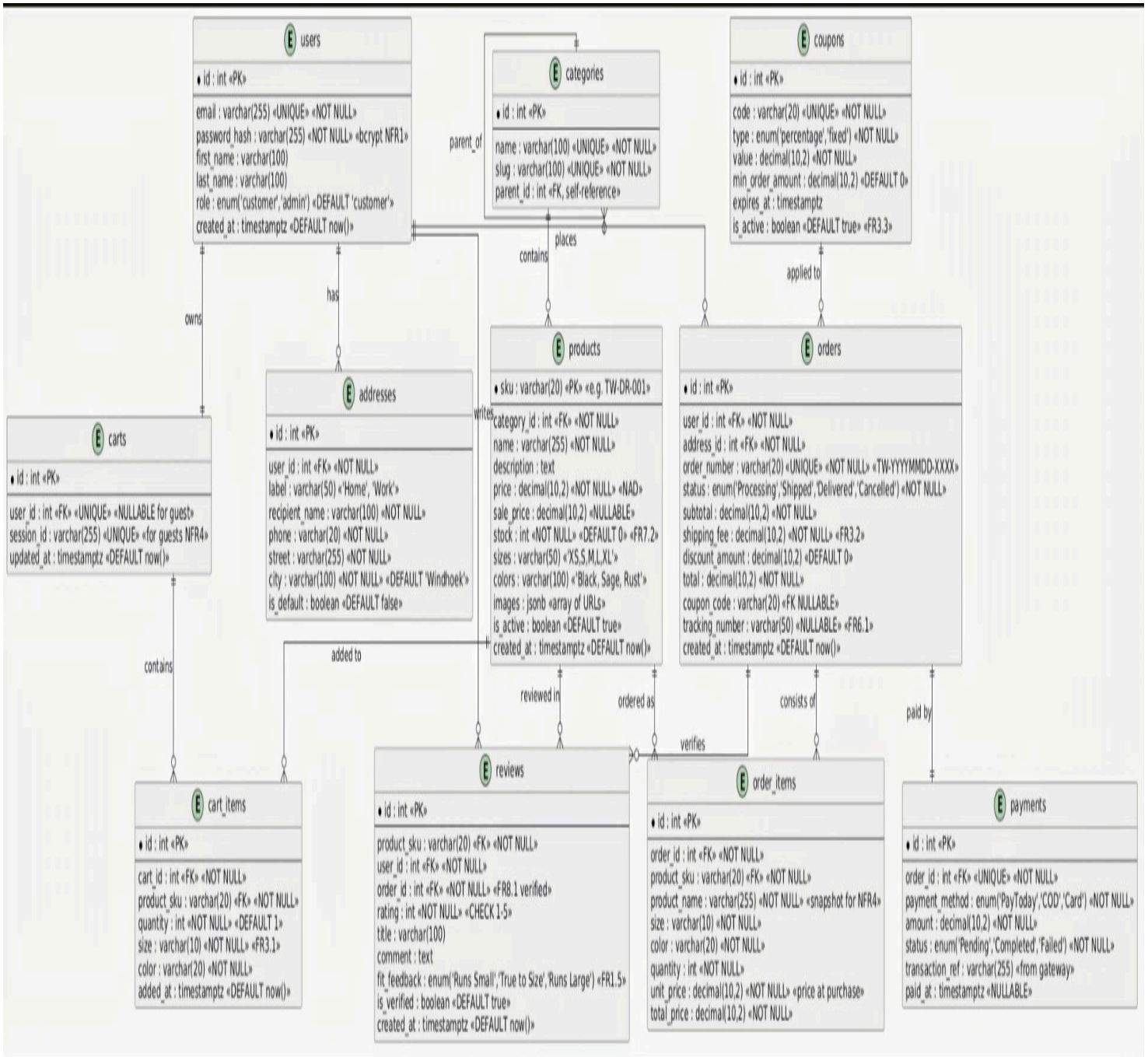


Figure 4: Entity Relationship Diagram for TrendWind Database. Supports all Must Have FRs and NFR5 Scalability from Assignment 1.

- Normalization: “The database is in 3NF. is intentionally denormalized to preserve historical data per NFR4.

```
@startu

' avoid problems with backarrow

skinparam linetype ortho


entity "users" as users {

  *id : int <<PK>>


  email : varchar(255) <<UNIQUE>> <<NOT NULL>>

  password_hash : varchar(255) <<NOT NULL>> <<bcrypt NFR1>>

  first_name : varchar(100)

  last_name : varchar(100)

  role : enum('customer','admin') <<DEFAULT 'customer'>>

  created_at : timestamptz <<DEFAULT now()>>

}


entity "addresses" as addresses {

  *id : int <<PK>>


  user_id : int <<FK>> <<NOT NULL>>

  label : varchar(50) <<'Home', 'Work'>>

  recipient_name : varchar(100) <<NOT NULL>>

  phone : varchar(20) <<NOT NULL>>

  street : varchar(255) <<NOT NULL>>

  city : varchar(100) <<NOT NULL>> <<DEFAULT 'Windhoek'>>

  is_default : boolean <<DEFAULT false>>

}


entity "categories" as categories {

  *id : int <<PK>>


  name : varchar(100) <<UNIQUE>> <<NOT NULL>>

  slug : varchar(100) <<UNIQUE>> <<NOT NULL>>

  parent_id : int <<FK, self-reference>>

}


entity "products" as products {

  *sku : varchar(20) <<PK>> <<e.g. TW-DR-001>>


  category_id : int <<FK>> <<NOT NULL>>

  name : varchar(255) <<NOT NULL>>

  description : text

  price : decimal(10,2) <<NOT NULL>> <<NAD>>

  sale_price : decimal(10,2) <<NULLABLE>>

  stock : int <<NOT NULL>> <<DEFAULT 0>> <<FR7.2>>

  sizes : varchar(50) <<'XS,S,M,L,XL'>>

  colors : varchar(100) <<'Black, Sage, Rust'>>

  images : jsonb <<array of URLs>>

  is_active : boolean <<DEFAULT true>>

  created_at : timestamptz <<DEFAULT now()>>

}


entity "carts" as carts {

  *id : int <<PK>>


  user_id : int <<FK>> <<UNIQUE>> <<NULLABLE for guest>>

  session_id : varchar(255) <<UNIQUE>> <<for guests NFR4>>

  updated_at : timestamptz <<DEFAULT now()>>

}


entity "cart_items" as cart_items {

  *id : int <<PK>>


  cart_id : int <<FK>> <<NOT NULL>>

  product_sku : varchar(20) <<FK>> <<NOT NULL>>

  quantity : int <<NOT NULL>> <<DEFAULT 1>>

  size : varchar(10) <<NOT NULL>> <<FR3.1>>

  color : varchar(20) <<NOT NULL>>

  added_at : timestamptz <<DEFAULT now()>>

}


entity "coupons" as coupons {

  *id : int <<PK>>


  code : varchar(20) <<UNIQUE>> <<NOT NULL>>

  type : enum('percentage','fixed') <<NOT NULL>>

  value : decimal(10,2) <<NOT NULL>>

  min_order_amount : decimal(10,2) <<DEFAULT 0>>

  expires_at : timestamptz

  is_active : boolean <<DEFAULT true>> <<FR3.3>>

}


entity "orders" as orders {

  *id : int <<PK>>


  user_id : int <<FK>> <<NOT NULL>>

  address_id : int <<FK>> <<NOT NULL>>

  order_number : varchar(20) <<UNIQUE>> <<NOT NULL>> <<TW-YYYYMMDD-XXXX>>

  status : enum('Processing','Shipped','Delivered','Cancelled') <<NOT NULL>>

  subtotal : decimal(10,2) <<NOT NULL>>

  shipping_fee : decimal(10,2) <<NOT NULL>> <<FR3.2>>

  discount_amount : decimal(10,2) <<DEFAULT 0>>

  total : decimal(10,2) <<NOT NULL>>

  coupon_code : varchar(20) <<FK NULLABLE>>

  tracking_number : varchar(50) <<NULLABLE>> <<FR6.1>>

  created_at : timestamptz <<DEFAULT now()>>

}


entity "order_items" as order_items {

  *id : int <<PK>>


  order_id : int <<FK>> <<NOT NULL>>

  product_sku : varchar(20) <<FK>> <<NOT NULL>>

  product_name : varchar(255) <<NOT NULL>> <<snapshot for NFR4>>

  size : varchar(10) <<NOT NULL>>

  color : varchar(20) <<NOT NULL>>

  quantity : int <<NOT NULL>>

  unit_price : decimal(10,2) <<NOT NULL>> <<price at purchase>>

  total_price : decimal(10,2) <<NOT NULL>>

}


entity "payments" as payments {

  *id : int <<PK>>


  order_id : int <<FK>> <<UNIQUE>> <<NOT NULL>>

  payment_method : enum('PayToday','COD','Card') <<NOT NULL>>

  amount : decimal(10,2) <<NOT NULL>>

  status : enum('Pending','Completed','Failed') <<NOT NULL>>

  transaction_ref : varchar(255) <<from gateway>>
```

```
  paid_at : timestamptz <<NULLABLE>>
}

entity "reviews" as reviews {
  *id : int <<PK>>

  product_sku : varchar(20) <<FK>> <<NOT NULL>>
  user_id : int <<FK>> <<NOT NULL>>
  order_id : int <<FK>> <<NOT NULL>> <<FR8.1 verified>>
  rating : int <<NOT NULL>> <<CHECK 1-5>>
  title : varchar(100)
  comment : text
  fit_feedback : enum('Runs Small','True to Size','Runs Large') <<FR1.5>>
  is_verified : boolean <<DEFAULT true>>
  created_at : timestamptz <<DEFAULT now()>>
}

' Relationships
users ||--o{ addresses : "has"
users ||--o{ orders : "places"
users ||--o{ reviews : "writes"
users ||--|| carts : "owns"
categories ||--o{ products : "contains"
categories ||--o{ categories : "parent_of"
products ||--o{ cart_items : "added to"
products ||--o{ order_items : "ordered as"
products ||--o{ reviews : "reviewed in"
carts ||--o{ cart_items : "contains"
orders ||--o{ order_items : "consists of"
orders ||--|| payments : "paid by"
orders ||--o{ reviews : "verifies"
coupons ||--o{ orders : "applied to"

@enduml
```

3.3.2 Entity Descriptions & A1 Traceability

Entity	Purpose	Key Attributes	A1 Requirements Supported
users	Stores customer + admin accounts	password_hash (bcrypt)	FR2.1, FR2.2, NFR1
products	Women's catalog (e.g., sku = TW-DR-001)	stock, sizes, sale_price	FR1.1, FR1.5, FR7.1, FR7.2, FR7.3
orders	Transaction header	status, tracking_number, total	FR4.1, FR4.2, FR4.3, FR6.1, FR3.2
order_items	Transaction line items	unit_price (snapshot)	NFR4 Reliability
payments	Payment record	payment_method, status	FR5.1, FR5.2, FR5.3
reviews	User-generated content	fit_feedback, is_verified	FR8.1, FR1.5
carts	Persistent shopping cart	session_id (for guests)	FR3.1, NFR4

3.3.3 Design Decisions for Scalability NFR5

Why as PK on : String PK allows human-readable IDs like from A1 Section 3.3.1. Indexed by default. Supports 10,000+ SKUs without performance loss.

Why snapshots + : If admin deletes TW-BT-007, historical orders remain valid. Critical for NFR4 and financial auditing.

Why has + : Enforces FR3.1. Unique constraint prevents duplicate rows.

Why FK: Enforces FR8.1 “verified purchasers only”. A user can only review a product if it exists with that .

Indexes for NFR2: Create indexes on , , , to ensure <1.5s search response.

3.3.4 Data Integrity Rules

Stock Control: on enforces FR7.2. in Figure 3 Sequence Diagram must be in a transaction.

PCI Compliance: table has NO column. Only from Stripe per NFR1.

COD Workflow: + for FR5.2. Admin updates on delivery.

4. Traceability Matrix

4.1 Purpose

This matrix demonstrates bi-directional traceability between functional and non-functional requirements specified in Assignment 1 and the design components in this SDD. It ensures all Must Have and Should Have requirements are addressed and provides a verification baseline for Assignment 3 Implementation and Assignment 4 Testing.

4.2 Functional Requirements Traceability

A1 ID	A1 Requirement Description	Priority	Design Component	Figure Ref	Design Justification
FR1.1	Display products in responsive grid with image, name, price, discount badge	Must Have	Product class, ProductService.getAll(), React ProductGrid component	Fig 1, Fig 2	Product entity stores price, sale_price. getDisplayPrice() method calculates badge.
FR1.2	Keyword search with autocomplete	Must Have	ProductService.search(), PostgreSQL ILIKE index on products.name	Fig 1	Indexed search meets NFR2 <1.5s response time.
FR1.3	Filter by Size, Color, Style, Material, Price	Must Have	Product attributes sizes, colors, ProductService.filter()	Fig 2, Fig 4	sizes, colors stored as CSV. API applies WHERE clauses.
FR1.4	Flash Sale with countdown timer	Should Have	Product.sale_price, React FlashSale component	Fig 1, Fig 2	sale_price ≠ NULL flags flash items. Timer is client-side.
FR1.5	Size guide with CM + fit feedback	Must Have	Review.fit_feedback, Product description	Fig 2, Fig 4	fit_feedback enum aggregated for size guide display.
FR1.6	Curve+Plus category 1XL–3XL	Must Have	Category = 'Curve+Plus', Product.sizes	Fig 4	Categories table allows filtering. sizes includes 1XL, 2XL, 3XL.
FR2.1	User registration email +	Must Have	User class, AuthService.register(), password_hash bcrypt	Fig 2, Fig 4	users table enforces UNIQUE email.

	password				NFR1 satisfied.
FR2.2	User login/logout	Must Have	AuthService.login(), JWT session	Fig 1	Stateless auth for SR1 100 concurrent users.
FR2.3	Save multiple delivery addresses	Should Have	Address entity, User 1--* Address	Fig 2, Fig 4	addresses.user_id FK. is_default flag.
FR2.4	Save items to wishlist	Could Have	Out of scope V1.0. Future: wishlist table	N/A	Deferred per A1 1.2 Scope.
FR3.1	Add items to cart, modify qty/size	Must Have	Cart, CartItem classes, CartService.addItem()	Fig 2, Fig 3, Fig 4	cart_items stores size, color. Unique constraint prevents duplicates.
FR3.2	Calculate subtotal, shipping, total	Must Have	OrderService.calculateTotal(), orders fields	Fig 3, Fig 4	Logic shown in Fig 3 step 11. shipping_fee = N$65 per A1 2.2.
FR3.3	Apply discount coupon codes	Must Have	Coupon entity, OrderService.applyCoupon()	Fig 2, Fig 4	coupons.is_active + expires_at validated.
FR4.1	Generate unique order record	Must Have	Order entity, order_number format TW-YYYYMMDD-XXXX	Fig 2, Fig 3, Fig 4	INSERT shown in Fig 3 step 12.
FR4.2	View order history + status	Must Have	Order entity, OrderService.getByUser()	Fig 2, Fig 4	orders.user_id FK enables history query.
FR4.3	Admin update order status	Must Have	Order.status enum, OrderService.updateStatus()	Fig 2	AdminDashboard calls API. Triggers FR6.2 email.

FR5.1	PayToday integration sandbox	Must Have	PaymentService.processPayToday(), Payment.method='PayToday'	Fig 1, Fig 2	External API call. Status tracked in payments.
FR5.2	Cash on Delivery option	Must Have	Payment.method='COD', status='Pending'	Fig 2, Fig 4	Order created immediately. Stock decreased per FR7.1.
FR5.3	Visa/Mastercard via PCI gateway	Should Have	PaymentService.processCard(), Stripe API, Tokenization	Fig 1, Fig 2, Fig 3	Fig 3 steps 15–16. No card data stored per NFR1.
FR6.1	Generate tracking number on Shipped	Should Have	Order.tracking_number, OrderService.generateTracking()	Fig 2, Fig 4	NamPost API called when status → 'Shipped'.
FR6.2	Email notification on status change	Should Have	NotificationService, triggered by Order.updateStatus()	Fig 1	Out of scope for Class Diagram. Shown in Architecture.
FR7.1	Decrease stock on payment confirm	Must Have	InventoryService.decreaseStock(), products.stock UPDATE	Fig 2, Fig 3, Fig 4	Fig 3 steps 20b–21b. Transaction-wrapped.
FR7.2	Prevent checkout for zero stock	Must Have	Product.checkStock(), CHECK (stock ≥ 0)	Fig 2, Fig 3, Fig 4	Fig 3 steps 4–7a. Throws exception.
FR7.3	Display “Low Stock: Only X left”	Should Have	Product.stock, React conditional render if stock < 5	Fig 2	ProductService returns stock count.
FR8.1	Verified purchasers submit reviews	Should Have	Review entity, order_id FK, is_verified	Fig 2, Fig 4	reviews.order_id ensures only buyers can review.

4.3 Non-Functional Requirements Traceability

A1 ID	A1 Requirement	Design Component	Figure Ref	Design Justification
NFR 1	Security: bcrypt, TLS 1.3, PCI-DSS, tokenization	User.password_hash, Stripe tokens, HTTPS in Architecture	Fig 1, Fig 2, Fig 3	password_hash uses bcrypt. Card data never stored. All API calls via HTTPS.
NFR 2	Performance: <2.5s page load, <1.5s search	PostgreSQL indexes on sku, category_id, name, Vercel CDN	Fig 1, Fig 4	3-Tier + CDN meets load time. Indexed queries meet search time.
NFR 3	Usability: <4min checkout, pinch-zoom	React SPA mobile-first, Product.images (jsonb)	Fig 1	Presentation Tier optimized per OE-3 and SR3.
NFR 4	Reliability: 30-day cart, daily backups	carts.session_id, Supabase automated backups	Fig 2, Fig 4	session_id persists guest carts. order_items snapshots data.
NFR 5	Scalability: 10,000 SKUs, 1,000/month	products table, PostgreSQL, Normalized 3NF design	Fig 4	sku PK indexed. 3NF prevents redundancy. Serverless scales per SR1.
SR1	100 concurrent users	Node.js non-blocking, Vercel Serverless, connection pooling	Fig 1	Application Tier scales horizontally.
SR2	99% uptime	Vercel + Supabase	Fig 1	Managed services

		SLA		exceed 99.9% uptime.
SR3	Responsive 360px–1920px	React + TailwindCSS, ProductGrid component	Fig 1	Presentation Tier implements CSS Grid/Flexbox.

4.4 Requirements Not Addressed in V1.0 Design

A1 Requirement	Reason	Future Version
FR2.4 Wishlist	Could Have priority. Out of scope A1 1.2	V2.1
AR Virtual Try-OnFR2.4 Wishlist	Out of scope A1 1.2	V2.0
Multi-vendor	Out of scope A1 1.2	V2.0
Men’s/Kids lines	Out of scope A1 1.2	V2.0

References:

Sommerville, I. (2016). Software Engineering (10th ed.). Pearson Education.

IEEE Std 1016-2009. IEEE Standard for Information Technology Systems Design Software Design Descriptions. IEEE Computer Society.

Fowler, M. (2003). UML Distilled: A Brief Guide to the Standard Object Modeling Language (3rd ed.). Addison-Wesley.

Richardson, C. (2018). Microservices Patterns: With Examples in Java. Manning Publications.

Stripe, Inc. (2025). Stripe API Reference: Charges. Retrieved from https://stripe.com/docs/api

PayToday Namibia. (2025). PayToday Merchant Integration Guide v2.1. Windhoek, Namibia.

Supabase. (2025). Supabase Documentation: PostgreSQL. Retrieved from https://supabase.com/docs

University of Namibia. (2025). COMP3721 Course Notes: Human-Computer Interaction & Design Patterns. Windhoek, Namibia.

PlantUML. (2025). PlantUML Language Reference Guide. Retrieved from https://plantuml.com/

Vercel Inc. (2025). Vercel Documentation: Serverless Functions. Retrieved from https://vercel.com/docs